

Project Report

On

Age-of-Information-Aware Digital Twin Synchronization in Vehicular Edge Networks

Submitted by

Name	**Roll No.**
Angadi Kanishka Kumar	262CS004
Sai Kaustav S	262CS026
Pillanagovilla Arvind	262IS022

Project Mentor: Anshul

Department of Computer Science and Engineering

National Institute of Technology Karnataka,
Surathkal

Academic Year 2026–2027

Abstract

In vehicular edge computing systems, a Digital Twin (DT) of each vehicle can be hosted at a nearby Road Side Unit (RSU). The usefulness of the twin depends on how recent the state received from the physical vehicle is. This report describes an implementation that studies this freshness using Age of Information (AoI). The SUMO traffic simulator is interfaced with Python through TraCI. The position, speed, heading and acceleration of each vehicle are sampled, passed through a simplified communication-channel model, and used to update a Digital Twin on the RSU side. Three synchronization schemes are considered: periodic, event-triggered, and AoI-aware adaptive. The experiments use five configured demand levels and a fixed simulation horizon of 120 seconds. The measures collected are AoI, state-estimation errors, packet delivery ratio and communication overhead. The results show the expected freshness–overhead trade-off: frequent periodic updates give lower AoI and position error but require more transmissions, while less frequent updates reduce overhead but leave the twin with older information. For the configuration tested, the adaptive approach gives a middle operating point. Because the channel is an abstract delay-and-loss model, the results should be read as a controlled simulation study rather than a full evaluation of a wireless network.

Keywords: Age of Information, Digital Twin, Vehicular Edge Computing, Roadside Unit, SUMO, TraCI, synchronization.

Contents

1 Introduction and Problem Understanding **1**

2 Objectives and Scope **2**

3 Tools and Technologies **3**

4 System Architecture and Workflow **4**

5 Synchronization Strategies **5**

5.1 Periodic Synchronization 5

5.2 Event-Triggered Synchronization 5

5.3 AoI-Aware Adaptive Synchronization 5

6 Experimental Setup **6**

7 Metrics and Evaluation Method **7**

8 Results and Discussion **8**

8.1 Average Age of Information 8

8.2 Average Position Error 9

8.3 Communication Overhead 9

8.4 Packet Delivery Ratio 10

8.5 Comparison at Configured Demand Level 100 10

9 Implementation Structure **12**

10 Reproducibility and Validation **13**

11 Limitations and Future Work **14**

12 Conclusion **15**

1 Introduction and Problem Understanding

A Digital Twin is a software representation of a physical entity that is updated through observations of that entity. In a vehicular edge setting, an RSU may keep a local representation of the vehicles in its vicinity. The representation may include properties such as position, speed, heading and acceleration. Applications that rely on this information may become less reliable when the twin is not updated frequently enough.

Age of Information is a simple metric of information freshness. For a vehicle, AoI is the time elapsed between the current simulation time and the generation time of the most recent state stored by the Digital Twin. AoI is not simply the time since the last packet arrived: it is based on when the state inside that packet was originally generated. Hence, packet delay or packet loss can increase the age of the information available at the RSU.

This project studies how the choice of synchronization policy affects freshness and communication cost. Sending updates very frequently keeps the twin up to date but increases the number of transmissions. Sending updates less frequently reduces communication overhead, but the stored state may become stale. The implementation compares three policies under the same traffic and channel assumptions.

2 Objectives and Scope

- Build a traffic-simulation environment using SUMO and read vehicle state through TraCI.
- Implement an RSU-side Digital Twin that stores the freshest successfully received state for each vehicle.
- Implement periodic, event-triggered and AoI-aware adaptive update policies.
- Model communication delay and packet loss in a repeatable software channel.
- Measure AoI, position error, speed error, heading error, packet delivery ratio and transmission overhead.
- Compare the policies at configured demand levels of 5, 10, 20, 50 and 100.

The scope is limited to a single simulated traffic environment and an abstract communication channel. The work does not implement a standards-compliant vehicular radio stack, multi-RSU handover, real vehicle hardware or a production deployment.

3 Tools and Technologies

Table 3.1: Tools and Technologies Used

Tool / Technology	**Role in the Project**
SUMO 1.18.0	Microscopic road-traffic simulation and vehicle movement generation.
TraCI	Python interface used to advance SUMO and read vehicle states during simulation.
Python 3	Experiment control, synchronization logic, channel model and metric calculation.
NumPy / Pandas	Numerical and tabular processing of experiment outputs.
Matplotlib	Generation of comparison plots from the experiment summary CSV.
Ubuntu Linux	Execution environment for SUMO and the Python project.

4 System Architecture and Workflow

The implementation follows an update path from the vehicle to the RSU. SUMO advances road traffic, and TraCI reads the current state of every active vehicle. The state is sent according to the chosen synchronization policy. The communication channel model either discards the attempted packet or queues it for delivery after a configurable delay. When a packet arrives at the RSU, the Digital Twin checks its generation timestamp and stores the state only if it is more recent than the currently stored state. The experiment runner logs AoI and state errors and writes summary metrics to a CSV file.

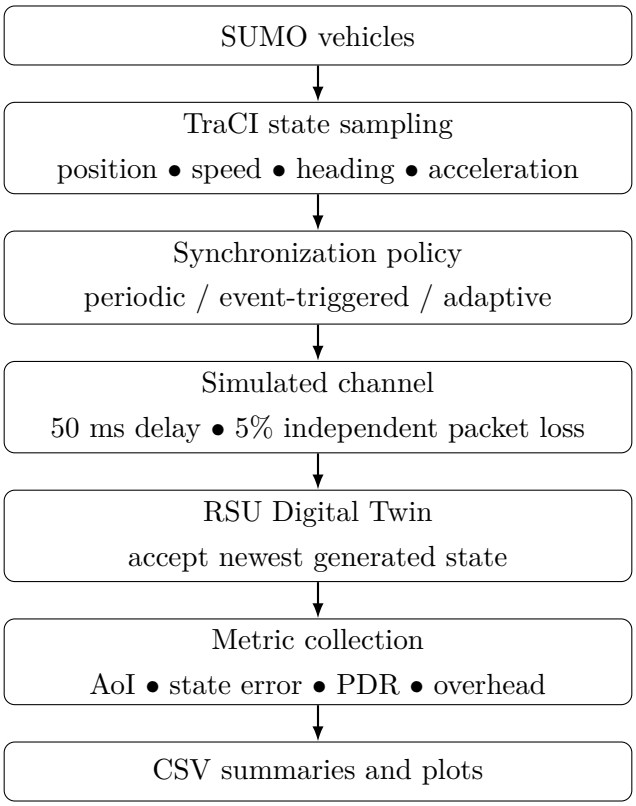


Figure 4.1: Processing Flow Implemented in the Experiment

5 Synchronization Strategies

5.1 Periodic Synchronization

Description: A vehicle sends an update at a fixed interval. Three intervals are evaluated: 50 ms, 100 ms and 500 ms. A shorter interval generally lets the twin receive newer state information more often, but it increases the number of transmission attempts.

5.2 Event-Triggered Synchronization

Description: After the first update, a new packet is attempted when a state change crosses at least one configured threshold: a position change of 5 m, a speed change of 2 m/s, a heading change of 15 degrees, or an acceleration change of 2 m/s^2. A 500 ms fallback interval also forces an attempt if no threshold event occurs. The fallback controls attempted transmissions; it does not guarantee packet reception.

5.3 AoI-Aware Adaptive Synchronization

Description: The adaptive policy selects an update interval using the AoI available at the twin and the current vehicle state. It selects 50 ms when AoI is unavailable, AoI is at least 0.20 s, or the absolute acceleration is at least 2 m/s^2. It selects 100 ms when AoI is above 0.05 s or speed is at least 10 m/s. Otherwise, it selects 500 ms. This rule gives shorter intervals to states considered more urgent by the implemented thresholds.

6 Experimental Setup

The road network is a connected 3×3 grid generated with SUMO `netgenerate`. The network validation reported 24 edges in the largest connected component, with 100% edge coverage. The traffic demand generator creates five configured demand levels: 5, 10, 20, 50 and 100. These values are scenario labels based on configured trip demand; they should not be interpreted as guaranteed simultaneous vehicle counts.

Table 6.1: Experimental Parameters

Parameter	**Setting**
SUMO simulation duration	120 seconds per experiment
Simulation step	0.05 seconds
Communication delay	0.05 seconds (50 ms)
Independent packet loss probability	0.05 (5%)
Packet size used for overhead	100 bytes per attempted update
Random seed	42
Configured demand levels	5, 10, 20, 50 and 100
Configurations per demand level	3 periodic + 1 event-triggered + 1 adaptive
Total experiment configurations	25

7 Metrics and Evaluation Method

- **Age of Information:** At simulation time t, AoI is the difference between t and the generation time of the newest state stored in the twin. Samples are included in the AoI statistics only after the twin has received a state for that vehicle.
- **Position Error:** Euclidean distance between the current SUMO position and the position stored in the twin.
- **Speed Error:** Absolute difference between the current and stored speed.
- **Heading Error:** Smallest angular difference, accounting for the $0°/360°$ boundary.
- **Packet Delivery Ratio (PDR):** Successfully delivered packets divided by transmission attempts, multiplied by 100.
- **Communication Overhead:** Attempted transmissions multiplied by the assumed 100-byte packet size, divided by the 120-second simulation duration. This is an estimated payload-byte rate, not a measurement of complete network-layer traffic.

The experiment runner also logs the 95th percentile and maximum AoI, maximum position error, average speed error, average heading error, transmission attempts, delivered packets and dropped packets. Packet accounting was checked for all 25 configurations; in each row, attempts equaled delivered plus dropped packets.

8 Results and Discussion

The experiment summary contains 25 configurations: 15 periodic runs (three intervals for each of the five demand levels), five event-triggered runs and five adaptive runs. All runs used the same simulation horizon of 120 seconds. The resulting plots compare average AoI, average position error, communication overhead and PDR against the configured demand.

8.1 Average Age of Information

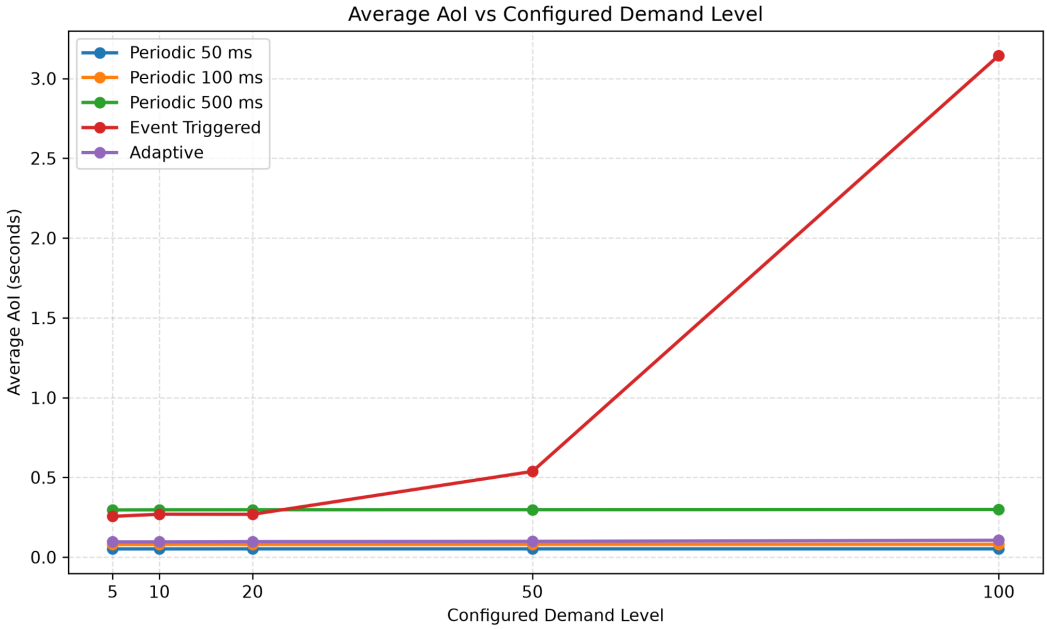


Figure 8.1: Average AoI against Configured Demand for the Evaluated Synchronization Strategies

8.2 Average Position Error

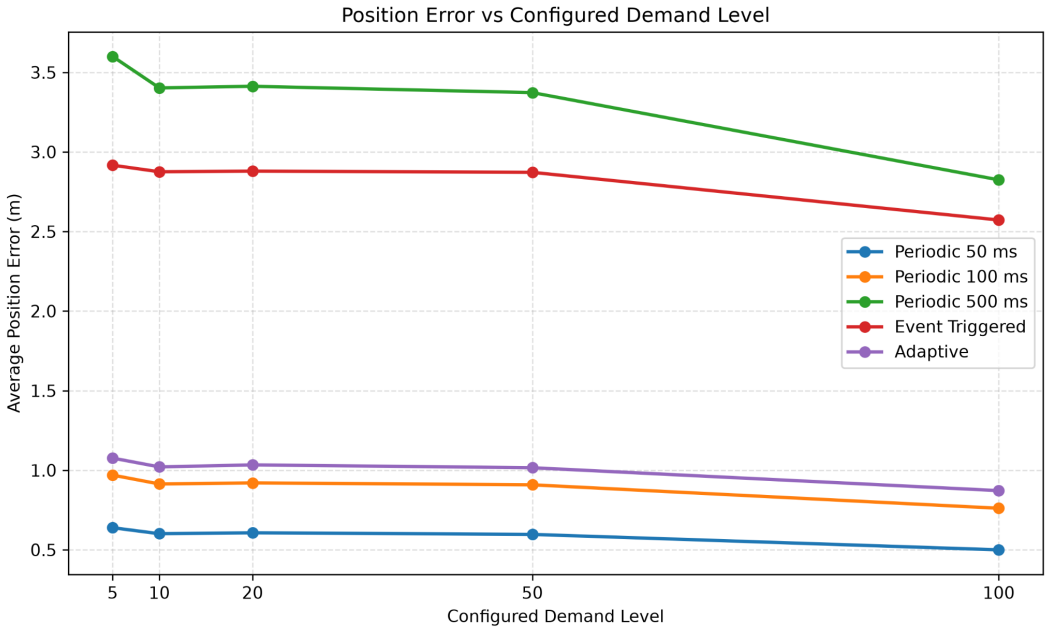


Figure 8.2: Average Position Error against Configured Demand

8.3 Communication Overhead

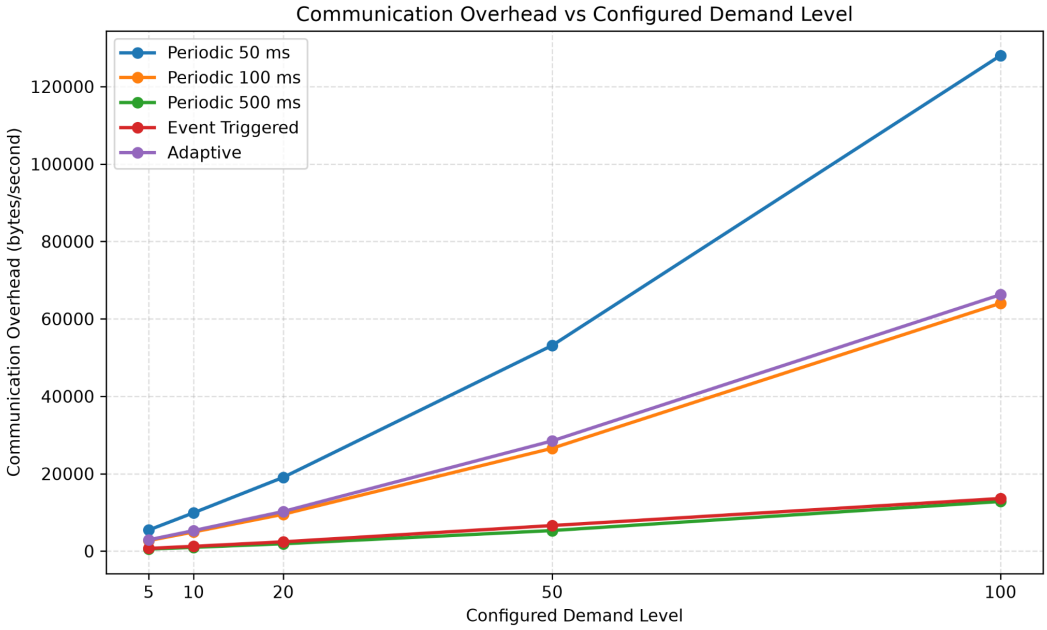


Figure 8.3: Estimated Communication Overhead against Configured Demand

8.4 Packet Delivery Ratio

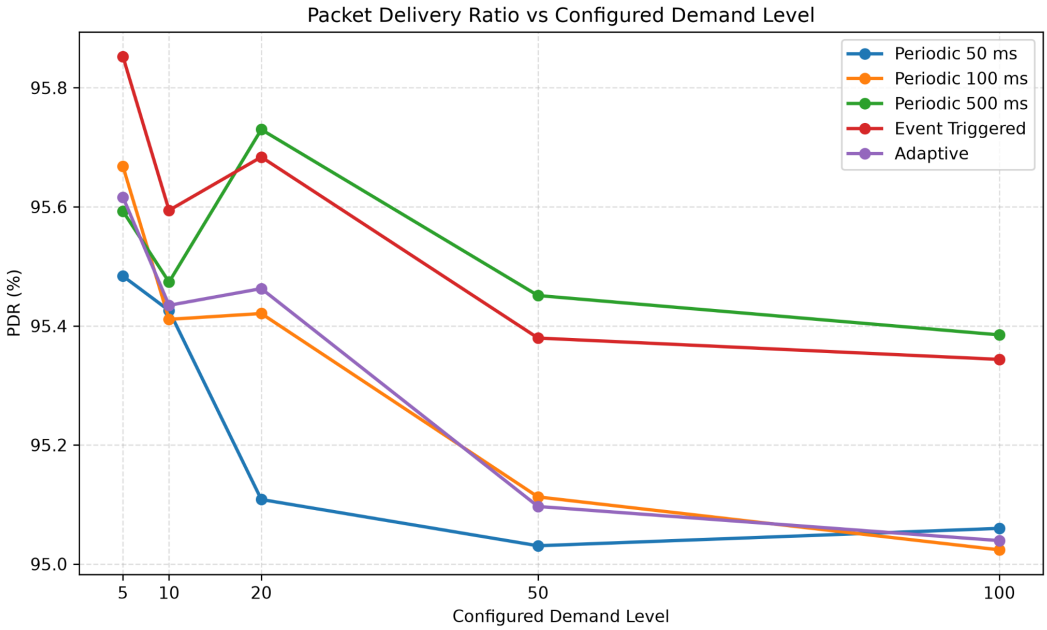


Figure 8.4: Packet Delivery Ratio against Configured Demand

8.5 Comparison at Configured Demand Level 100

The following values are taken from the final experiment summary for configured demand 100. They illustrate the trade-off between information freshness and transmission cost in this scenario.

Table 8.1: Comparison of Strategies at Configured Demand 100

Strategy	Avg. AoI (s)	Avg. Pos. Error (m)	Overhead (bytes/s)
Periodic, 50 ms	0.052600	0.5061	93,295
Periodic, 100 ms	0.080227	0.7709	46,662.5
Periodic, 500 ms	0.299135	2.8561	9,353.33
Event-triggered	0.258079	2.3632	11,832.5
Adaptive	0.109630	0.8984	48,022.5

At demand 100, the 50 ms periodic policy had the lowest average AoI and position error among the configurations listed, but it also generated the highest overhead. The 500 ms periodic policy had the lowest overhead but the highest average AoI and position error. The adaptive policy lay in between: its average AoI and position error were significantly

lower than those of the 500 ms periodic and event-triggered policies, while its overhead was still far lower than that of the 50 ms periodic policy. In this run, event-triggered synchronization reduced overhead compared with the 100 ms periodic policy, but its average AoI and position error were higher.

PDR remained at about 95–96% across the experiments, in line with the configured independent packet-loss probability of 5%. The event-triggered policy can still experience an AoI peak above its 500 ms fallback interval, because the fallback interval determines when a transmission is attempted, not when it is successfully delivered. Packet loss and channel delay can increase the time until a new state arrives at the twin.

In the generated dataset, the observed position error does not increase monotonically with configured demand. The report therefore does not claim that higher demand necessarily causes larger error. The plotted values should be understood as arising from the particular traffic scenario and random seed used.

9 Implementation Structure

Table 9.1: Implementation Modules and Their Responsibilities

Module / File	Responsibility
`rsu_twin.py`	Stores the freshest successfully received vehicle state, generation time and reception time; computes AoI.
`metrics.py`	Computes position, speed and heading errors.
`channel.py`	Models independent packet loss and fixed delivery delay; maintains packet counters and pending arrivals.
`strategy_channel_experiment.py`	Runs one selected synchronization strategy, samples SUMO state, delivers packets and writes summary metrics.
`density_experiment.py`	Generates demand scenarios and orchestrates the 25 experiment configurations.
`plot_density_results.py`	Reads the summary CSV and creates the four comparison plots.

10 Reproducibility and Validation

The seed of the channel loss process is fixed to 42, and the same scenario settings are used for every run. A repeated event-triggered run was compared with the earlier event-triggered result, and the selected summary metrics were identical. We also checked that the final dataset satisfied attempts = delivered + dropped for all 25 rows of packet accounting. The runner reported a stop time of 120.00 seconds for all completed experiments.

These checks confirm repeatability with the same software version, scenario files, seed and parameters. They do not show that the results are valid for other road layouts, traffic patterns, radio models or random seeds.

11 Limitations and Future Work

- The communication channel uses a fixed 50 ms delay and independent 5% packet loss. It does not model radio propagation, interference, contention, bandwidth sharing, retransmissions or a full vehicular communication protocol.
- The assumed packet size is 100 bytes for overhead estimation; protocol headers and other network costs are not included.
- The road environment is a connected 3×3 grid, and the traffic scenarios represent a limited set of configured demand levels.
- The adaptive policy uses fixed thresholds, which have not been optimized through a separate training or parameter-search process.
- Future work can use a more realistic network simulator or wireless interface, multiple RSUs, handover, varied road layouts, multiple seeds and confidence intervals.

12 Conclusion

In this project, three approaches for synchronizing a vehicular Digital Twin with vehicle state generated in the SUMO simulator were implemented and evaluated. The twin on the RSU side stores the freshest successfully delivered state and computes AoI from the time that state was generated. The experiments show the expected trade-off between information freshness and communication overhead. In the demand-100 comparison, the highest freshness was achieved by frequent periodic updates, but this also consumed the most communication resources. Sending fewer updates reduced overhead but increased AoI and state error. Under the current decision rules, the adaptive method was a compromise between the two. The results are repeatable for the tested setup and are limited by the assumptions of the simplified communication model. The implementation provides a basis for further evaluation under more realistic vehicular networking conditions.